Assembly Language

Computer Organization

Chandra Gummaluru

chandra-gummaluru.github.io



assembly language

a readable form of the instructions

a processor runs

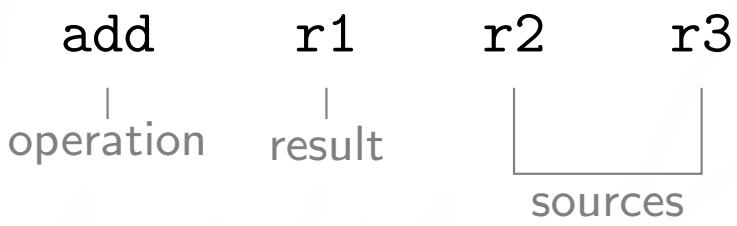
A processor runs instructions encoded as bits. Assembly language gives each instruction a short name so people can read and write it.

`00100001` becomes `add r1, r2, r3`

one line of assembly is one machine instruction

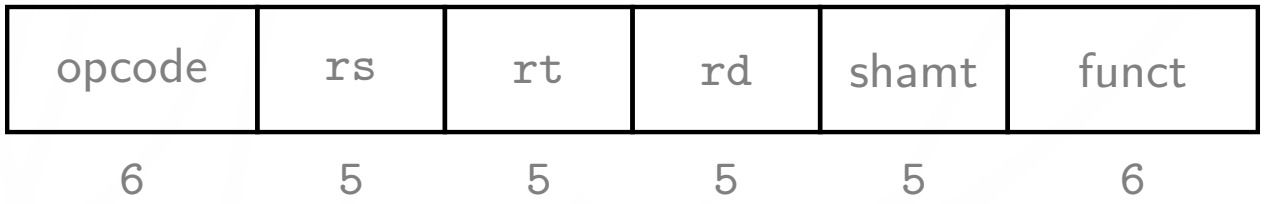
instruction format

An instruction names an **operation** and the **operands** it acts on.
Operands are usually registers, the small fast storage inside the processor.



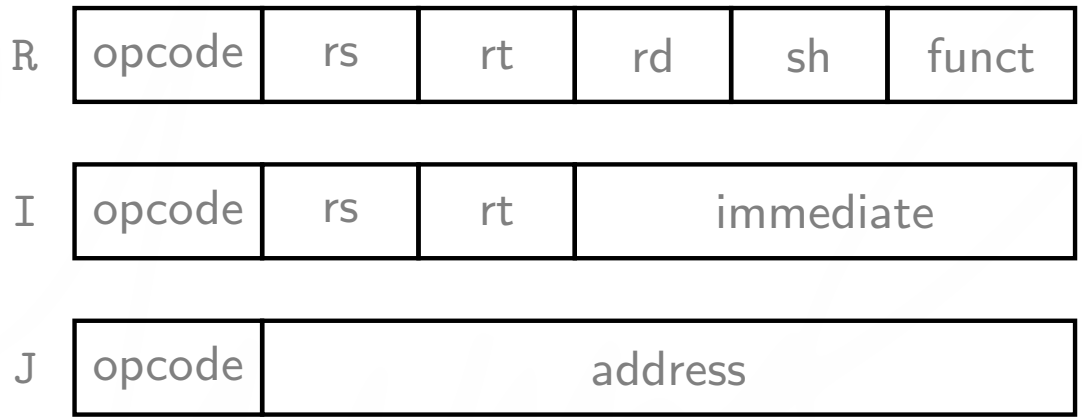
add the values in `r2` and `r3`, and place the result in `r1`

In this course we use the **MIPS** instruction set. Each instruction is exactly 32 bits, split into fields.



the opcode says what to do, the other fields say which registers to use

There are three formats. They share the opcode but differ in what follows.



register operations, operations with a constant, and jumps

Registers have names like `$t0` and `$t1`. A real MIPS add reads two and writes a third.

```
add $t0, $t1, $t2
```

$$\texttt{\$t0} = \texttt{\$t1} + \texttt{\$t2}$$

kinds of instructions

Most instructions fall into three groups.

- **arithmetic and logic**

  compute on registers, such as `add` and `sub`

- **data movement**

  copy between registers and memory, such as `load` and `store`

- **control flow**

  change which instruction runs next, such as `jump` and `beq`

a program

A **label** marks a position in the program. A control-flow instruction can jump to it, which is how we build loops.

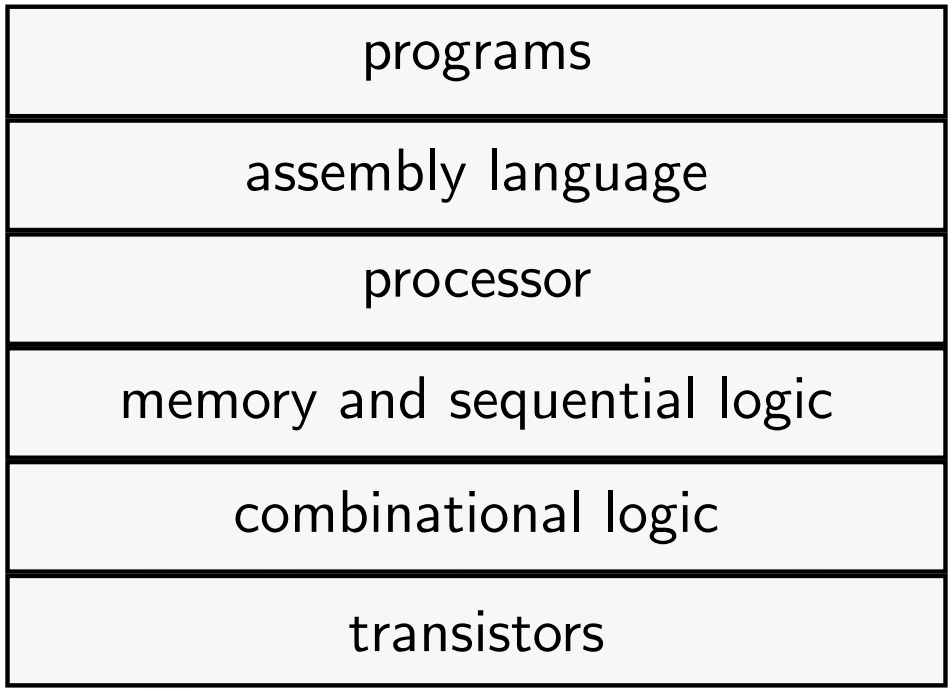
```
        li    r1, 0          ; sum = 0
        li    r2, 10         ; i = 10
loop:   beq   r2, r0, done   ; if i == 0, leave the loop
        add   r1, r1, r2     ; sum = sum + i
        sub   r2, r2, 1      ; i = i - 1
        jump  loop           ; repeat
done:   store r1, total      ; save the result
```

This short program adds the numbers from 10 down to 1. It shows the whole idea of a program:

- **registers** hold the values being worked on
- **labels and branches** decide what runs next
- **memory** keeps the final result

the whole stack

We have built every layer, from the bottom to the top.



A program is just a great many transistors switching in step. That is the whole idea of computer organization.